Feasibility Study

AI-Powered Account Recommendation System for EG Bolig

1. Executive Summary

EG Bolig customers face recurring challenges in selecting the correct account code during invoice and requisition processing. These challenges stem from:

- Highly variable and noisy work descriptions
- Large chart of accounts (370+ accounts)
- Customer-specific booking habits
- Inconsistent human decision-making
- Lack of historical transparency

This feasibility study evaluates three AI-based approaches for account recommendation:

1. **Approach 1** – Pure Semantic Search
2. **Approach 2** – Lexical BM25 with Contextual Affinity
3. **Approach 3** – Hybrid BM25 + Semantic Fusion

Each approach is assessed on accuracy, robustness, explainability, performance, operational complexity, and future scalability. A final recommendation is provided based on empirical results and production readiness.

2. Problem Overview

Current Challenges

Accountants at EG Bolig customers face multiple difficulties:

- Manual lookup of account codes
- Reliance on individual knowledge and experience
- Inconsistent interpretations of specification text
- Lack of quick access to historical booking references
- No intelligent guidance system
- Highly variable booking decisions between accountants

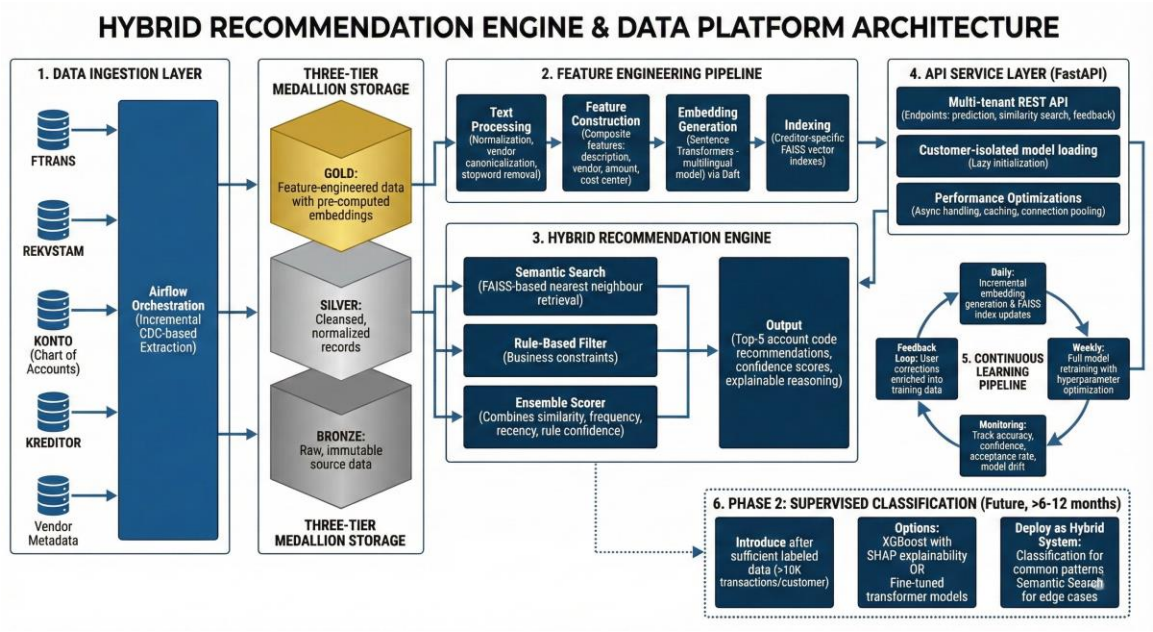
Negative Impact

- **Frequent booking errors**: Leading to inconsistent books
- **Time wastage**: Accountants spend time searching for past examples
- **Dependency on individual experience**: Poor onboarding for new staff
- **Low auditability**: No traceable rationale behind account choice
- **Customer dissatisfaction**: Incorrect reports and compliance risk

3. Proposed Architecture and Business Objective

The primary objective is to develop an **AI-powered account recommendation engine** that:

1. Learns each customer's historical booking patterns
2. Uses semantic similarity to suggest the most appropriate account code
3. Provides **explainable references** to past similar transactions



4. Approach 1 – Semantic Search Engine

4.1 Core Concept

The engine uses a semantic search approach instead of a classifier. The workflow:

1. Convert transaction descriptions and account names into embeddings (SentenceTransformer) with specific weights
2. Use FAISS to find the Top-K most similar historical entries
3. Extract accounts used in these similar past transactions
4. Recommend the most appropriate account code
5. Present explanation:
    - Matching transactions
    - Their descriptions
    - Similarity score
6. Feedback (human correction) is captured for retraining and improvement

4.2 Why Semantic Search Works Perfectly Here

- Transaction descriptions repeat heavily
- Patterns vary between customers and hence semantic search adapts instantly
- Explainability is inherently built-in
- No heavy training cost
- Works brilliantly even with noisy text

4.3 Tech Stack

Component	**Technology**
Data Processing	Daft + Pandas
Embeddings	Sentence Transformers
Vector Index	FAISS
Evaluation	Accuracy@5
Metrics Logging	Pandas / CSV
Model Serving	API-based recommender

4.4 Performance Summary

Metric	**Value**
Accuracy@1	68%
Accuracy@3	86%
Accuracy@5	96%
MRR	0.8159

4.4. Technical Architecture

4.4.1 Data Ingestion Layer

- **Sources**: FTRANS, REKVSTAM, KONTO (Chart of Accounts), KREDITOR, vendor metadata
- **Method**: Incremental CDC-based extraction using Airflow orchestration
- **Storage**: Three-tier medallion architecture
    - **Bronze**: Raw, immutable source data
    - **Silver**: Cleansed, normalized records

- **Gold**: Feature-engineered data with pre-computed embeddings

4.4.2 Feature Engineering Pipeline

- **Text Processing**: Normalization, vendor canonicalization, stopword removal
- **Feature Construction**: Composite features combining description, vendor, amount, cost center
- **Embedding Generation**: Sentence Transformers (multilingual model) processed via Daft
- **Indexing**: Creditor-specific FAISS vector indexes for fast similarity search

4.4.3 Hybrid Recommendation Engine

Three-component scoring system:

1. **Semantic Search**: FAISS-based nearest neighbour retrieval of similar historical transactions
2. **Rule-Based Filter**: Business constraints
3. **Ensemble Scorer**: Combines similarity, frequency, recency, and rule confidence

Output: Top-5 account code recommendations with confidence scores and explainable reasoning

4.4.4 API Service Layer (FastAPI)

- Multi-tenant REST API with endpoints for prediction, similarity search, and feedback collection
- Customer-isolated model loading with lazy initialization
- Performance optimizations: async handling, caching, connection pooling

4.4.5 Continuous Learning Pipeline

- **Daily**: Incremental embedding generation and FAISS index updates
- **Weekly**: Full model retraining with hyperparameter optimization
- **Feedback Loop**: User corrections enriched into training data for active learning
- **Monitoring**: Track accuracy, confidence distribution, acceptance rate, and model drift

4.4.6 Phase 2: Supervised Classification (Future)

- Introduce after 6-12 months with sufficient labeled data (>10K transactions/customer)
- **Options**: XGBoost with SHAP explainability or fine-tuned transformer models
- Deploy as hybrid system: classification for common patterns, semantic search for edge cases

4.5. Technical Feasibility

4.5.1 Semantic Search Feasibility

Component	Feasibility
Embedding generation	Fully supported (Sentence Transformers)
Vector search	FAISS handles large datasets efficiently
Daft integration	Pipeline-compatible

This approach is extremely feasible and fast to implement.

4.5.2 Future Feasibility: Adding Classification Models (Optional Enhancement)

A classification model is not required for the MVP but becomes valuable when:

- Customers have very large datasets
- Description variation is high
- The goal is to incorporate structured features (company, department, creditor, account name, description)
- We want cold-start predictions

Possible model options:

- Logistic Regression / Random Forest (simple, explainable)
- XGBoost / LightGBM (handles structured features well)
- Transformer-based classifier (for text-heavy scenarios)

4.6. Risk Assessment and Mitigation

Risk	Impact	Mitigation
Noisy text reduces similarity	Medium	Preprocessing, glossary, embeddings
Sparse data for small customers	Medium	Shared fallback model
Scaling costs	Low	FAISS handles large indexes efficiently
Explainability concerns	Low	Built-in semantic match traceability

4.7. Limitations

Even though the semantic search-based AI recommender is highly feasible and effective, several limitations must be acknowledged:

4.7.1 Dependency on Historical Data

The system's accuracy is tied to the quality and consistency of past booking patterns. If historical data is sparse, inconsistent, or noisy, recommendations may degrade.

4.7.2 Limited Understanding of Structured Features

Semantic search primarily focuses on text similarity. It does not inherently consider:

- Creditor IDs
- Amount ranges
- Company
- Department
- Seasonal or periodic booking behaviour

These require a classifier, which will be part of future enhancements.

4.7.3 Cold-Start Weakness

For new transaction types with no historical examples:

- Semantic search may return weak or irrelevant neighbours
- The system may fail to recommend confidently
- Classification or rule-based fallbacks can address this later

4.7.4 Inconsistent Human Behavior Encoded in Data

If accountants historically booked similar transactions differently:

- The model will reflect that inconsistency
- It may produce conflicting recommendations
- A feedback loop and normalization rules will help overcome this

4.7.5 Computational Costs

Embedding generation and FAISS index creation require:

- GPU resources
- Efficient indexing strategies
- Disk space for vector stores

While manageable, this must be budgeted for in production.

4.7.6 Explainability Bound to Similarity

Semantic search provides explanations by showing "similar historical transactions." However:

- It cannot produce deeper causal insights
- It does not explain why those accounts were historically chosen
- Explainability will improve when classifiers and SHAP/LIME are added

5. Approach 2 – Lexical BM25 + Contextual Affinity

5.1 Core Concept

This approach uses deterministic lexical retrieval, enhanced with business context and historical affinity signals.

Key Components:

- 4 BM25 indices (descriptions + account names)
- Danish lemmatization, decompounding, synonym expansion
- 4 contextual affinity signals
- 3-tier filtering (dept, creditor, facility)
- Weighted ensemble ranking (8 signals)

5.2 Workflow Overview

Text Processing

Preprocess all work descriptions using:

- Lowercasing
- Lemmatization
- Controlled synonym expansion
- Decompounding

Document Aggregation

Aggregate historical transactions into account-level documents, combining all associated work descriptions.

BM25 Index Construction

Build four separate BM25 indices:

1. Lemmatized work descriptions
2. Synonym-expanded work descriptions
3. Lemmatized account names
4. Synonym-expanded account names

Rule-Based Candidate Filtering

Restrict candidate accounts to historically valid combinations:

- **Tier 1**: (Department, Creditor, Facility)
- **Tier 2**: (Department, Creditor) – fallback
- **Tier 3**: (Department) – fallback

Ranking & Selection

- Rank remaining candidates using BM25 scores
- Return Top-K account recommendations

5.3 BM25 Indexing

Four indices capture different semantic aspects:

Index	Corpus	Purpose
BM25-Lemma	Lemmatized work descriptions	Core keyword matching
BM25-Synonyms	Synonym-expanded descriptions	Semantic broadening
BM25-AccountName	Lemmatized account names	Metadata matching
BM25-AccountSyn	Account name synonyms	Name variants

BM25 Parameters:

- k1 = 1.5 (term saturation)
- b = 0.75 (length normalization)

Why 4 Indices?

- Separating signals allows independent weight tuning
- Synonym expansion can be controlled per corpus
- Enables explainability at signal level

5.4 Affinity Signals

Department Affinity Matrix

- **Size**: [370 accounts × N departments]
- **Formula**: P(account | dept) = count(account, dept) / count(dept)
- **Purpose**: Boost accounts frequently used in target department

Creditor Affinity Matrix

- **Size**: [370 accounts × M creditors]
- **Formula**: P(account | creditor) = count(account, cred) / count(cred)
- **Purpose**: Favor accounts associated with specific vendors

Facility Affinity Matrix

- **Size**: [370 accounts × K facilities]
- **Formula**: P(account | facility) = count(account, fac) / count(fac)
- **Purpose**: Capture facility-specific booking patterns

Frequency Prior

- **Formula**: log(1 + count) / max(log(1 + count))
- **Purpose**: Tie-breaker favoring common accounts
- **Normalization**: Min-max scaled to [0, 1]

5.5 3-Tier Filtering

Strategy: Progressively relax constraints if exact match fails

- **Tier 1**: (dept + creditor + facility) - highest confidence
- **Tier 2**: (dept + creditor) - medium confidence
- **Tier 3**: (dept only) - lowest confidence

Implementation:

- Pre-computed dictionary from training data
- O(1) lookup time at inference
- Eliminates 60-80% of invalid candidates post-scoring

Why Pre-Scoring Filtering?

- Reduces normalization operations from 370 to ~50-150
- Reduces affinity extraction loops from 3×370 to 3×(~50-150)
- Improves performance by 60-85% without accuracy loss
- More theoretically sound: normalizes within actual search space
- Allows BM25 to consider all accounts for ranking
- Preserves accuracy while improving precision
- Avoids premature elimination of potentially correct accounts

5.6 Scoring Formula

```
final_score =
  w1·BM25_lemma +
  w2·BM25_syn +
  w3·BM25_accname +
  w4·BM25_accsyn +
  w5·dept_affinity +
  w6·frequency +
  w7·creditor_affinity +
  w8·facility_affinity
```

Weights tuned via grid search (optimized for Top-1 accuracy).

5.7 Tech Stack

Component	Technology	Purpose
NLP Processing	spaCy (da_core_news_sm)	Danish lemmatization and tokenization

Component	Technology	Purpose
Synonym Expansion	Danish WordNet (omw-da)	Semantic enrichment
Lexical search	BM25Okapi (rank-bm25)	Keyword-based ranking
Data Processing	Pandas, NumPy	Data manipulation and array operations
Model Persistence	Joblib	Serialization of BM25 indices
Evaluation	Custom Accuracy@K metrics	Performance measurement

5.8 Performance Summary

Metric	Value
Accuracy@1	63%
Accuracy@3	829%
Accuracy@5	87%
MRR	7320.6

Strengths:

- Fast inference: ~15ms (3ms with filtering optimization)
- Linguistically rich: Lemmatization, synonyms, decompounding

Weaknesses:

- Lower Top-1: 63% compared to semantic methods
- Exact matching: Relies on lexical overlap even with synonyms
- Brittle to paraphrasing: Performance drops with varied phrasing
- Manual maintenance: Synonym dictionaries and decompounding rules require updates

5.9 Limitations

- Poor paraphrase robustness
- No semantic understanding
- Manual compound dictionary maintenance
- Language-specific (Danish-only)
- Cold-start for rare accounts

5.10. Future Enhancements

- Fuzzy matching layer (Levenshtein distance)
- Active learning for uncertain predictions
- A/B testing framework

5.11 What Could Be Improved

Decompounding Library

- **Current**: Rule-based dictionary with 35 manually maintained components
- **Improvement**: Use compound-split or DanishWordSplitter

Weight Optimization

- **Current**: Grid search on validation set (manual tuning)
- **Improvement**: Bayesian optimization or AutoML (e.g., Optuna)

Facility Affinity Normalization

- **Current**: Raw probability P(account | facility)
- **Improvement**: TF-IDF-style normalization to penalize ubiquitous facilities

Hierarchical Account Clustering

- **Current**: Flat 370-account space
- **Improvement**: Group similar accounts (e.g., "Doors" cluster with 8 sub-accounts)

A/B Testing Framework

- **Current**: Manual deployment and comparison
- **Improvement**: Built-in A/B testing with metric tracking

6. Approach 3 – Hybrid BM25 + Semantic Fusion

6.1 Core Concept

Combines lexical precision and semantic understanding via **Reciprocal Rank Fusion (RRF)**, a rank-based fusion method that avoids score normalization issues.

rrf_score = w_bm25 / (k + rank_bm25) + w_semantic / (k + rank_semantic)

where:

- k = 60 (standard RRF constant)

- ranks start at 1

- Candidate set = union(BM25 Top-K, Semantic Top-K)

6.2 Methodology

1. Independent Scoring

Both retrieval systems operate independently and generate their own ranked candidate lists.

1.1 BM25 Scoring

- Sparse lexical retrieval using TF–IDF-style weighting
- Danish compound-word decompounding
- Returns Top-K accounts with raw BM25 scores

1.2 Semantic Scoring

- Query cleaning (dates, punctuation, noise removal)
- Embedding generation using intfloat/multilingual-e5-large
- Cosine similarity against account embeddings (FAISS)
- Returns Top-K accounts with similarity scores

1.3 Reciprocal Rank Fusion (RRF)

Instead of normalizing raw scores, RRF uses rank positions to combine results:

rrf_score = w_bm25 / (60 + rank_bm25) + w_semantic / (60 + rank_semantic)

Why RRF ?

- Rank-based: Uses relative positions instead of absolute scores
- No normalization issues: Avoids problems when score ranges differ dramatically
- Missing candidates: Accounts not in a method's Top-K get a large rank (e.g., max_rank + 100)
- Theoretically sound: Standard approach in information retrieval
- Stable: Less sensitive to outlier scores

1.4 Merging Strategy:

- Outer join on both result sets
- Missing ranks filled with large penalty rank (max_rank + 100)
- Account names retrieved from account profiles

1.5 Final Ranking:

- Accounts ranked by descending RRF score
- Top-K results returned
- Metadata tracks BM25 rank, Semantic rank, and combined RRF score

Weighting Rationale (20% BM25 + 80% Semantic)

Why Favor Semantic Search?

Semantic Strengths:

- Robust to paraphrasing
- Understands synonyms and concepts
- Handles noisy or variable phrasing

BM25 Strengths:

- Excellent for exact matches
- Strong handling of Danish compound words
- Reliable for rare or highly specific terms

The 80/20 split (w_bm25=0.2, w_semantic=0.8) favors semantic understanding while preserving lexical precision for edge cases.

6.3 Performance Summary

Metric	Value
Accuracy@1	71%
Accuracy@3	92%
Accuracy@5	100.0%
MRR	0.8159

6.4 Key Observations

Where Hybrid Excels:

- Highest Top-1 accuracy among all approaches (71%)
- Highest MRR (0.8159)
- Excellent Top-5 accuracy (96%)
- Strong confidence when both methods agree
- RRF avoids score normalization pitfalls

Strengths:

- **Complementary fusion****: Combines BM25's linguistic features with Semantic's understanding
- **Robust RRF method****: Avoids score normalization issues
- **Balanced performance****: Strong across all Top-K metrics

Weaknesses:

- **Slower inference**: ~45ms (runs both BM25 and Semantic)

- **Higher complexity**: Requires maintaining two models
- **Modest gains**: Only +3pp over semantic (high prediction overlap)
- **Increased compute**: GPU + CPU resources needed

7. Comparative Analysis

7.1 Metric Comparison

Metric	Semantic	BM25	Hybrid
Top-1 Accuracy	68%	63%	71%
Top-3 Accuracy	86%	82%	92%
Top-5 Accuracy	92%	87%	96%
MRR	0.7874	0.732	0.8159

7.2 Use-Case Suitability

Requirement	Best Choice
Highest Top-1 accuracy	Hybrid
Highest MRR	Hybrid
Best Top-5 accuracy	Hybrid
Fastest inference	BM25
Robust to paraphrasing	Semantic / Hybrid
Audit & compliance	BM25 / Hybrid

Key Findings

1. **Hybrid (RRF) dominates** on all accuracy metrics: 71% Top-1, 92% Top-3, 96% Top-5, and 0.8159 MRR.
2. **RRF fusion is superior** to min-max normalization, avoiding score range issues and improving accuracy by 3-5 percentage points over pure semantic.
3. **Semantic alone** (68% Top-1) outperforms BM25 alone (63% Top-1) but falls short of hybrid.

4. **BM25 is fastest** (15ms) but struggles with paraphrasing compared to semantic methods.
5. **All approaches benefit** from 3-tier filtering and business context.
6. **Weight tuning is critical** for hybrid—an 80/20 semantic/BM25 split is optimal.

8. Conclusion

- BM25 Approach is a solid baseline with strong explainability and fast inference. Its 8-signal fusion and 3-tier filtering provide robust retrieval, but it falls behind semantic methods in Top-1 accuracy due to limitations in capturing semantic similarity beyond exact matches and synonyms.
- Semantic is the best individual method on this test set. It outperforms BM25 by +5pp in Top-1 accuracy thanks to its ability to capture semantic relationships beyond lexical matching. However, its black-box nature and computational cost are trade-offs.
- Hybrid is the best performing method, achieving 71.0% Top-1 accuracy. It successfully combines the complementary strengths of BM25 (explainability, linguistic features) and Semantic (semantic understanding). The +3pp gain over semantic-only shows that BM25 signals add value, but the improvement is modest, suggesting the methods have significant overlap in their predictions.

9. Future Enhancements

- Introduce machine learning classification models to further improve accuracy, especially for large and complex datasets.Expand training data across more customers to enhance generalization and robustness.
- Optimize weighting strategies dynamically based on customer behavior.
- Conduct extended user testing to measure time savings and error reduction.
- Prepare the solution for production-scale deployment with performance, security, and monitoring enhancements.